ASSIGNMENT 9.1

Lo que le falta a una API que ya responde

Microejercicios de serialización, errores, OpenAPI y tests sobre las demos

DURACION	MODALIDAD	PREREQUISITOS
4 x 5 min en la clase de 2 h	Individual, durante la clase 1	API funcional de S8; Python 3.12+ y uv

1. Encargo

Observar en demos FastAPI ejecutables los cuatro huecos de una API que ya responde: salida sin contrato, errores con formas distintas, documentación que no refleja la API y ningún test que avise si el contrato cambia.

Regla de continuidad
No se escribe una API nueva: se ejecutan, observan y modifican temporalmente las demos de examples/; la Wine Quality API de S8 no se toca hasta la clase 2.

2. Punto de partida

- Cada microejercicio cierra un bloque de teoría: M1 serialización, M2 errores, M3 OpenAPI y M4 tests.

- Las demos son deterministas y docentes: ningún score representa una decisión real de crédito ni de calidad.

- Lo que observes hoy es el requisito previo directo del workshop de la clase 2.

3. Material que recibes

- examples/serialization_demo.py, errors_demo.py y openapi_demo.py: una app FastAPI de un archivo por bloque

- tests/test_examples.py con TestClient, parametrize y dependency_overrides

- exercises/01-api-microexercises/problem/README.md con las preguntas de M1-M4

- Entorno del módulo con uv.lock; /docs, /redoc y /openapi.json en http://127.0.0.1:8000

4. Tareas

01 **Prepara el entorno** - Desde semana9/modules/09-api-professional sincroniza con uv sync --locked y confirma con uv run pytest -q que las demos parten con 9 tests en verde.

02 **M1 - Serialización** - Con serialization_demo.py compara /v1/scores/leaky y /v1/scores/public; anota los campos que solo aparecen en la primera, explica por qué /v1/scores/filtered tampoco los devuelve aunque la función retorna un dict y mira en /docs si PublicScore incluye is_reliable.

03 **M2 - Errores** - Con errors_demo.py envía a POST /v1/scores los tres cuerpos del enunciado y llama a GET /v1/nada y GET /v1/boom; tabula status, error.code y error.fields. Comenta el decorador de handle_validation, observa la respuesta por defecto y restáuralo.

04 **M3 - OpenAPI** - Con openapi_demo.py localiza en /openapi.json dónde terminan summary, docstring y responses de create_score; ejecuta el ejemplo minor con Try it out y añade un tercer ejemplo no_income sin el campo income.

05 **M4 - Tests** - En tests/test_examples.py localiza el test con dependency_overrides, quita temporalmente raise_server_exceptions=False del fixture errors_client y explica qué cambia; escribe un test para income: 0.

06 **Restaura y verifica** - Deshaz los cambios temporales de M2 y M4; si conservas el ejemplo de M3, adapta el test que cuenta ejemplos. Deja pytest y Ruff en verde con tu test incluido.

5. Entrega

Evidencia	Que debe permitir comprobar
M1	Campos filtrados y una frase sobre qué hace response_model con lo que no declara.
M2	Tabla de las cinco respuestas de error y respuesta por defecto sin el manejador.
M3	Ubicación en /openapi.json y desplegable de Swagger con tres ejemplos.
M4	Explicación del cambio en el fixture y salida de pytest con tu test en verde.

6. Criterios de aceptacion

[ ] La respuesta de M1 menciona response_model y qué hace con los campos que no declara.

[ ] Las cinco respuestas de error de M2 tienen la clave error en la raíz y ninguna contiene TU_SECRETO.

[ ] handle_validation queda restaurado tras observar el {"detail": ...} por defecto de FastAPI.

[ ] Swagger muestra tres ejemplos y la suite sigue en verde, o el cambio de M3 se deshace.

[ ] El test nuevo de M4 comprueba 422 y fields[0].field == "income" para income: 0.

[ ] uv run pytest -q y uv run ruff check . terminan en verde desde la raíz del módulo.

7. Rubrica orientativa

Criterio	Puntos
M1 - Serialización y filtrado	2
M2 - Sobre de error y fallos seguros	3
M3 - OpenAPI fiel a la API	2
M4 - Test nuevo y overrides	2
Restauración y reproducibilidad	1

8. Comprobacion

```
# Terminal 1 - demos
cd semana9/modules/09-api-professional
uv sync --locked
uv run fastapi dev examples/serialization_demo.py
# M2: examples/errors_demo.py   M3: examples/openapi_demo.py

# Terminal 2 - M2 con curl (Bash o Git Bash)
BASE_URL=http://127.0.0.1:8000
curl -i -X POST "$BASE_URL/v1/scores" \
  -H "Content-Type: application/json" \
  -d '{"age": "treinta", "income": 1000}'
curl -i "$BASE_URL/v1/nada"
curl -i "$BASE_URL/v1/boom"

# Terminal 2 - M4 y comprobación final
cd semana9/modules/09-api-professional
uv run pytest -q
uv run ruff check .
```

9. Notas y limites

- No escribas una API nueva ni toques la Wine Quality API: es el trabajo de la clase 2.

- Cambia de demo con Ctrl+C; las tres escuchan en el puerto 8000.

- PowerShell: usa curl.exe o lanza las peticiones desde /docs.

- La solución comentada se comparte al final de la clase.

Referencia en el repositorio

semana9/modules/09-api-professional/exercises/01-api-microexercises/problem/README.md
semana9/modules/09-api-professional/examples/README.md
semana9/modules/09-api-professional/sessions/01-teoria-api-profesional/README.md